

Lab8: Servo Motor Control with Microcontroller

Key Concepts:

This lab will help you understand the fundamentals of generating PWM signals using STM32F0 and controlling servo motors, laying the groundwork for more advanced motion control projects in embedded systems.

Goals

- Understand how to interface and control a Micro SG90 servo motor with STM32F0 (Blue Pill).
 - Learn how to generate PWM (Pulse Width Modulation) signals to control the servo motor's angle.
 - Rotate the servo motor to specific angles based on inputs or pre-defined logic.
 - Optionally, integrate user input from a keypad or sensor readings to control the motor's angle.
-

Grading Rubrics (Total = 20 points, with 2 bonus points)

1. **Documentation and Maintainability (5 points)**
2. **Functionality and Correctness (5 points)**
3. **Lab Demonstration (10 points)**
4. **Extra features (2 points)**

Lab Preparation

- **Read:** STM32 GPIO, PWM generation using Timers, and Interrupts Handling from Textbook
- **Datasheets:** Micro SG90 Servo Motor.
- **Tools:** STM32CubeIDE or Keil uVision.
- **MCU:** STM32F0 (Blue Pill).

Lab Assignment:

Part 1: Control Servo Motor Using PWM

Servo Motor Overview:

- **Micro SG90** is a servo motor that operates by receiving PWM signals to control its angle.
- **Operating voltage:** 4.8-6.0V (VCC to 5V).
- **Angle control:** PWM signal controls the position of the motor shaft. A pulse of 1ms corresponds to 0°, 1.5ms to 90°, and 2ms to 180°.
- **Pins:**
 - **VCC** → 5V.
 - **GND** → Ground.
 - **Signal** → PWM pin (controlled by MCU).

Steps:

1. **Connect the Servo Motor to STM32F0:**
 - **VCC** → 5V.
 - **GND** → Ground.
 - **Signal** → PA6 (or any available GPIO pin configured for PWM output).
2. **PWM Signal Setup:**
 - Use the STM32F0 Timer to generate PWM signals.
 - Set the timer to have a period corresponding to a 20ms cycle (50Hz frequency).
 - Adjust the duty cycle (width of the pulse) to control the servo angle.
3. **Servo Angle Control:**
 - A pulse width of **1ms** corresponds to **0°**.
 - A pulse width of **1.5ms** corresponds to **90°**.
 - A pulse width of **2ms** corresponds to **180°**.

Code Example:

```
#include "stm32f0xx_hal.h"

#define SERVO_PIN GPIO_PIN_6
#define SERVO_PORT GPIOA

TIM_HandleTypeDef htim3; // Timer handle for PWM

// Initialize PWM
void Servo_PWM_Init(void) {
    // Configure the timer for PWM output
    TIM_OC_InitTypeDef sConfigOC = {0};

    htim3.Instance = TIM3;
    htim3.Init.Prescaler = 47; // Adjust based on system clock (for 1μs resolution)
    htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim3.Init.Period = 20000 - 1; // 20ms period (50Hz frequency)
    htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    HAL_TIM_PWM_Init(&htim3);

    // Configure PWM output on channel 1
    sConfigOC.OCMode = TIM_OCMODE_PWM1;
    sConfigOC.Pulse = 1500; // Initial position (90 degrees)
    sConfigOC.OCpolarity = TIM_OCPOLARITY_HIGH;
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
    HAL_TIM_PWM_ConfigChannel(&htim3, &sConfigOC, TIM_CHANNEL_1);

    // Start PWM
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
}

// Function to control the servo angle (0° to 180°)
void Servo_Set_Angle(uint8_t angle) {
```

```
// Convert the angle (0-180) to corresponding pulse width (1000-2000 microseconds)
uint32_t pulse_width = 1000 + (angle * 1000) / 180;
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pulse_width); // Update PWM duty cycle
}
```

In this code:

- **Servo_PWM_Init()** initializes the timer and PWM with a 20ms period (50Hz) and sets an initial duty cycle of 1.5ms for 90°.
- **Servo_Set_Angle()** takes an angle input (0° to 180°) and converts it to the appropriate PWM pulse width (1ms to 2ms).

Part 2: Predefined Rotation Logic

In this part, you will rotate the servo motor to specific angles (e.g., 0°, 90°, 180°) based on certain conditions or timed intervals.

```
void Rotate_Servo_Predefined(void) {
    Servo_Set_Angle(0); // Rotate to 0 degrees
    HAL_Delay(1000);    // Wait for 1 second

    Servo_Set_Angle(90); // Rotate to 90 degrees
    HAL_Delay(1000);    // Wait for 1 second

    Servo_Set_Angle(180); // Rotate to 180 degrees
    HAL_Delay(1000);    // Wait for 1 second
}
```

This code rotates the servo motor between 0°, 90°, and 180° at 1-second intervals.

Part 3: Servo Control via Keypad or Sensor Input (Optional)

Optionally, add an input method (e.g., keypad or sensor) to control the servo motor's angle dynamically:

- **Keypad input:** Use a 4x4 keypad to input the desired angle and rotate the servo.
- **Sensor-based control:** Use an ultrasonic sensor to rotate the servo based on distance.

```
void Control_Servo_With_Keypad(void) {
    uint8_t angle = Keypad_Get_Angle(); // Read the desired angle from the keypad
    Servo_Set_Angle(angle);              // Set the servo to the corresponding angle
}
```

CSE331 Lab 8 Lab Demo

1. **Connect the hardware:** Ensure proper connections between the STM32F0, servo motor, and any input devices (if used).
 2. **Predefined angles:** Demonstrate the servo motor rotating to predefined angles (0°, 90°, and 180°).
 3. **Optional:** Show dynamic control of the servo motor using a keypad or sensor input.
-

Questions for Lab Report:

1. How would you modify your code to make the servo rotate back and forth continuously (e.g., from 0° to 180° and back to 0°)?
2. What changes would you need to make to control multiple servo motors simultaneously?
3. Provide a control flow diagram for the servo motor control system.

Bonus Task:

Develop a system that rotates the servo motor to follow a moving object using distance measurements from an ultrasonic sensor.